Identifying Key Entities in Recipe Data

NLP Assignment Report

Named Entity Recognition using Conditional Random Fields

Assignment submitted as part of UpGrad NLP Module

1. Problem Statement

Online cooking platforms and meal-planning apps are growing rapidly. A major bottleneck in scaling these platforms is the manual tagging of recipe data - identifying which words represent ingredients, quantities, and measurement units. This is time-consuming and error-prone when done by hand. The goal of this project is to build a Named Entity Recognition (NER) system that can automatically classify each token in a recipe into one of three categories: ingredient, quantity, or unit. This enables structured recipe databases, better search functionality, and dietary filtering.

2. Dataset Overview

The dataset is a JSON file containing 285 recipe records. Each record has two fields:

- "input" - raw ingredient text for one recipe (space-separated tokens)
- "pos" - corresponding NER labels for each token (ingredient / quantity / unit)

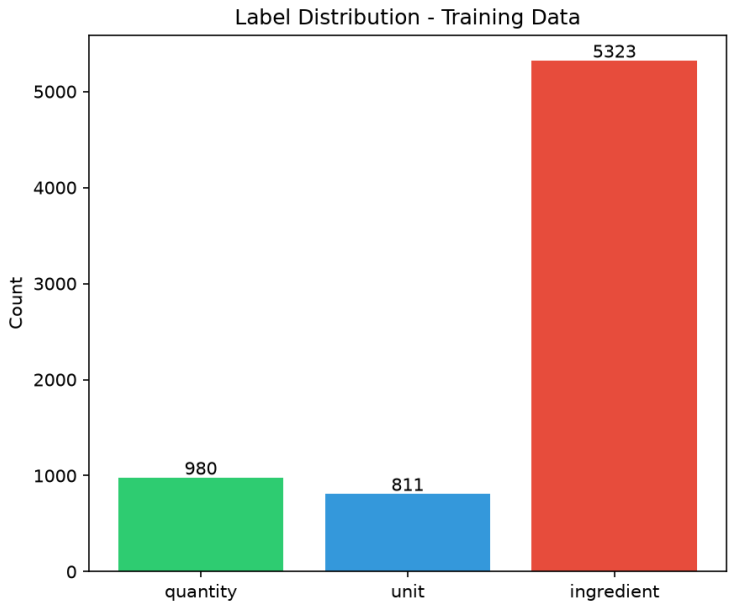
Data Quality Issues Found

During validation, I found that 5 rows (indices [17, 27, 79, 164, 207]) had a mismatch between the number of input tokens and the number of POS labels. For example, index 17 had 15 input tokens but only 14 labels. Since we cannot reliably use these for training, all 5 rows were dropped. After cleaning, 280 recipes remained. No null values or duplicates were present.

Label Distribution (Training Set)

After a 70:30 train-validation split (train=196, val=84), the training set had 7114 tokens distributed as:

- quantity: 980 tokens (13.8%)
- unit: 811 tokens (11.4%)
- ingredient: 5323 tokens (74.8%)

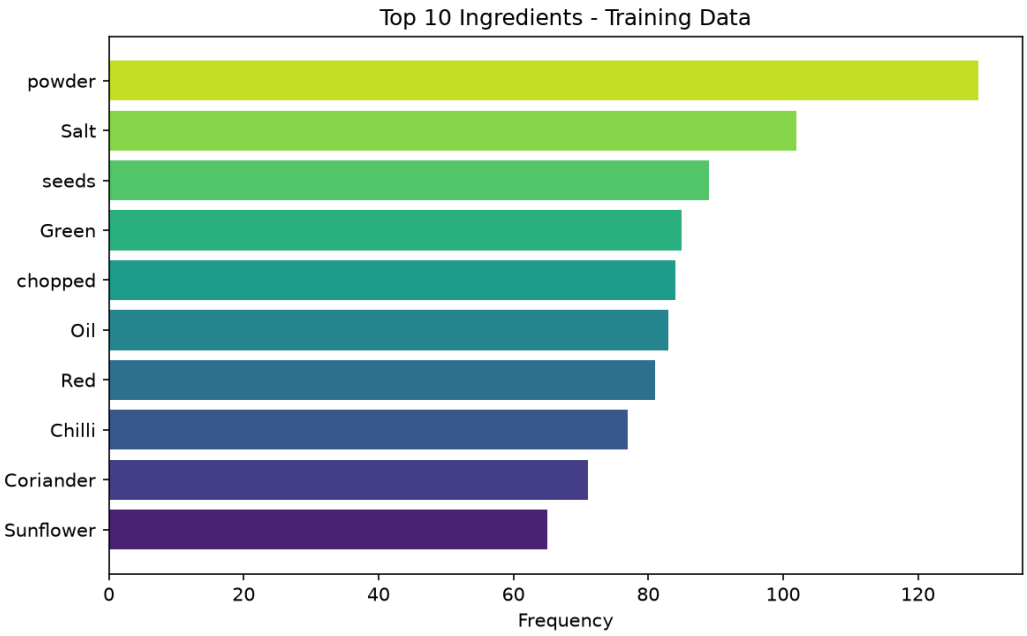


The "ingredient" label is heavily dominant (~74%), which creates a class imbalance problem. This was addressed later through inverse frequency weighting.

3. Exploratory Data Analysis

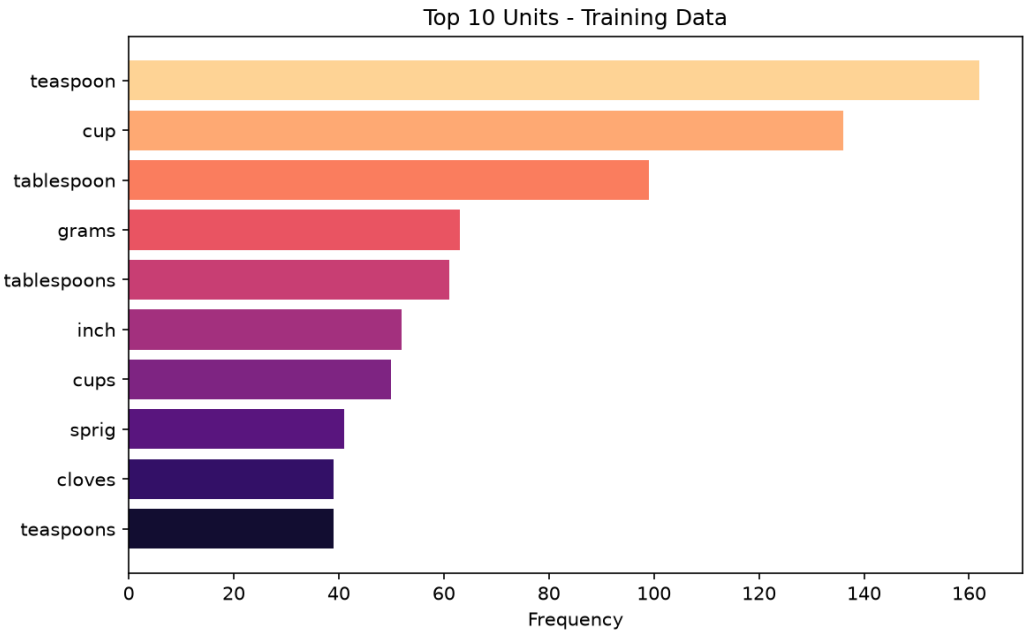
Top 10 Ingredients (Training)

The most frequent ingredients are common kitchen staples like Salt, Oil, Sugar, Ghee, and Cumin. This makes sense given that the dataset consists mostly of Indian recipes where these base ingredients appear in almost every dish.



Top 10 Units (Training)

"cup" and "teaspoon" are the dominant measurement units, followed by "tablespoon" and "cups". Most home cooking recipes use these standard volumetric measures rather than weight-based units.



4. Methodology

4.1 Approach Overview

The overall pipeline follows these steps:
1. Load and clean JSON data, tokenize input and labels
2. Split into 70:30 train-validation sets
3. Perform EDA to understand token distribution
4. Extract features for each token using a custom word2features function
5. Compute class weights and apply them to features
6. Train a CRF model using sklearn-crfsuite
7. Evaluate on training and validation sets
8. Perform error analysis on misclassified tokens

4.2 Feature Engineering

The word2features function extracts three categories of features for each token:

- Core Features: token text, lemma, POS tag, dependency relation, shape, and boolean flags (is_digit, is_stop, is_title, is_upper, has_hyphen, has_slash, is_punctuation)
- Quantity/Unit Detection: regex-based matching for numbers, fractions, and decimals. Keyword sets for common units (cup, tsp, gram, etc.) and quantity words (half, quarter, etc.)
- Contextual Features: previous and next token values, whether neighbors are quantities/units, BOS/EOS markers for sentence boundaries

spaCy (en_core_web_sm) was used to get linguistic features like lemma, POS tags, and dependency relations for each token.

4.3 Class Weighting

Since "ingredient" tokens outnumber quantity and unit tokens by roughly 5-7x, I used inverse frequency weighting to balance the classes. Additionally, the ingredient weight was further halved (penalised) to reduce bias towards the majority class.

Final class weights:

- quantity: 7.2592
- unit: 8.7719
- ingredient: 0.6682

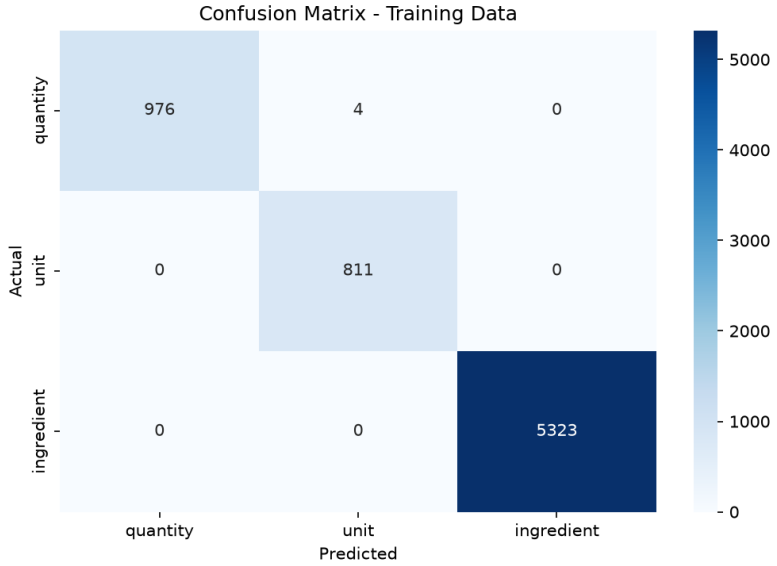
4.4 CRF Model Configuration

The CRF model was configured with:
- Algorithm: L-BFGS (quasi-Newton optimisation)
- L1 regularisation (c1): 0.5 (controls feature sparsity)
- L2 regularisation (c2): 1.0 (prevents overfitting)
- Max iterations: 100
- All possible transitions: True

5. Results

5.1 Training Performance

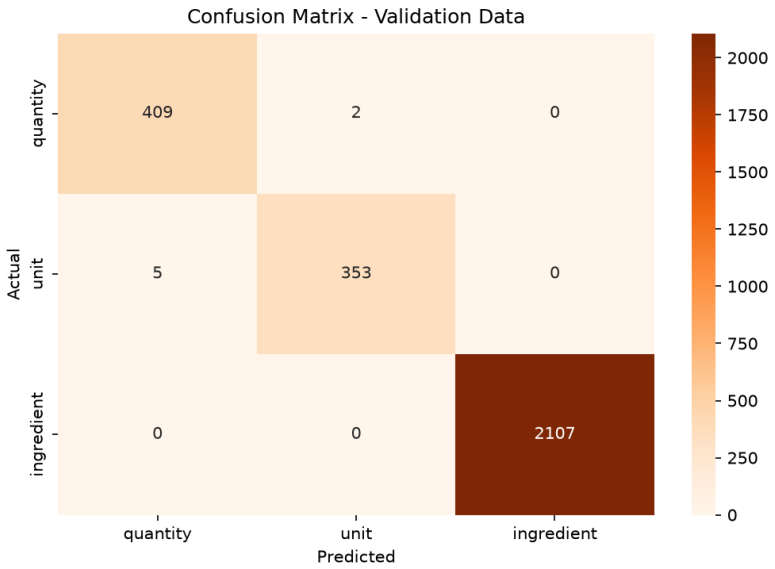
	precision	recall	f1-score	support
quantity	1.0000	0.9959	0.9980	980
unit	0.9951	1.0000	0.9975	811
ingredient	1.0000	1.0000	1.0000	5323
accuracy			0.9994	7114
macro avg	0.9984	0.9986	0.9985	7114
weighted avg	0.9994	0.9994	0.9994	7114



The model achieves near-perfect performance on training data (99.94% accuracy). Only a handful of quantity tokens were misclassified. This indicates the feature set is rich enough to capture the patterns in the data.

5.2 Validation Performance

	precision	recall	f1-score	support
quantity	0.9879	0.9951	0.9915	411
unit	0.9944	0.9860	0.9902	358
ingredient	1.0000	1.0000	1.0000	2107
accuracy			0.9976	2876
macro avg	0.9941	0.9937	0.9939	2876
weighted avg	0.9976	0.9976	0.9976	2876



On the validation set, the model achieves 99.76% accuracy with only 7 misclassified tokens out of 2876. The F1 scores for all three classes are above 0.99, showing strong generalisation.

6. Error Analysis

Out of 2876 validation tokens, only 7 were misclassified. Analysing these errors revealed the following patterns:

- Most errors occur between quantity and unit labels. A few quantity tokens get predicted as units and vice versa. This typically happens when a number appears next to an ambiguous word that could be either a unit or an ingredient.
- Ingredient classification is nearly perfect (100% recall). The penalised weighting successfully prevented the model from over-predicting ingredients.
- Boundary tokens (at the transition between quantity->unit->ingredient sequences) are the most error-prone positions. Adding more contextual window features could help here.
- The low error count (7 tokens) makes it hard to draw strong statistical conclusions, but the patterns observed are consistent with known CRF limitations at label transitions.

7. Assumptions

- Whitespace tokenisation is sufficient for this dataset (no complex tokenisation needed)
- The 5 rows with token-label mismatches are data entry errors and safe to drop
- spaCy en_core_web_sm provides adequate linguistic features for Indian recipe text
- Inverse frequency weighting is a reasonable approach for handling class imbalance
- The 70:30 split with random_state=42 gives a representative validation set

8. Conclusion

The CRF-based NER model successfully extracts ingredients, quantities, and units from recipe text with 99.76% accuracy on unseen data. The combination of linguistic features (via spaCy), keyword-based detection, regex patterns for numeric values, and contextual features from neighboring tokens proved effective for this structured prediction task.

For future work, the model could be improved by:
- Expanding the unit/quantity keyword sets with more regional cooking terms
- Adding character-level n-gram features for better handling of unseen words
- Trying different regularisation parameters via cross-validation
- Incorporating word embeddings as additional features
- Testing on a larger and more diverse recipe dataset